

1. Elvish languages of Middle-Earth

- `developed`: if we can infer an alphabetical order from the manuscript
- `incomplete`: if we cannot.

1.1 Extracting relationships of characters from the manuscript

Goal

- read the manuscript and extract the alphabetical relationship between the words and list the relationship of each character inferred.
- there is no "right" ordering at this moment; we need to figure out the right ordering from the output of this algorithm
- input: nested array of raw manuscript `$M$`
 - `$M$`: nested array; `M[i]` consists of the character array of word `$i$`.

```
[
  [ ` $ ` , ` % ` , ` % ` , ` * ` ] ,
  ...
]
```

- output: graph adjacency list `$G$` as a nested hash table

Pseudocode

```
ExtractCharRel(M) :
```

```
    G <- empty hash table // initialize the output
```

```
/* Main Loop: loop through the entire nested array (in wo
```

```
W: the number of words in the array
```

```
c: the same constant length of each word
```

```
L: the number of distinct characters in the manuscript
```

```
However, since each word has the same constant number of  
*/
```

```
    for word in range(W-1):
```

```
        checked <- false // checker if we already have th
```

```
        for char in range(c):
```

```
            if char not in G: // insert $L$ distinct char
```

```
                G[char] <- true
```

```
            if M[word][char] != M[word+1][char] AND check
```

```
                prereq <- M[word][char]
```

```
                following <- M[word+1][char]
```

```
                G[prereq][following] <- true
```

```
                checked <- true
```

```
    return G
```

Runtime Analysis

1. $O(1)$: output initialization
2. $O(Wc)$: main loop. We iterate through every single character in each word in nested array. In fact, we assume that the length of each word is the same constant c , we can simplify the Big O notation as $O(W)$.
3. $O(L)$: since we have to worry about L distinct characters in the manuscript, we have to execute INSERT operation L times to G . However, since $L \leq Wc$, we can ignore this in the final runtime analysis.
4. $O(1)$: simple `if` condition to check if the words are in alphabetical order.

Therefore, $O(1) + O(Wc) + O(L) + O(1) = O(W)$

Proof of correctness

For the algorithm to be correct, there are two things to be proven:

1. There must be L distinct vertices in G .
2. the edges between two vertices in G must be directed in ascending order. In other words, the character comparison between words must be from left to right.
3. L distinct vertices in G
 - the `if` statement to check if each character is in G before inserting edges (alphabetical relationships between characters of words) ensures that the nested hash table G has L distinct vertices that possibly include singleton nodes that are disconnected
4. edges in ascending order
 - the main loop that iterates through Wc times where W is the number of words in the manuscript and c is the constant number of characters of each character checks the alphabetical ascending order with `if` statements
 - suppose there two words of $M[word]$ and $M[word+1]$ that we compare to get the ascending order. To check the order of each character from the words in ascending order, `M[word][character] != M[word+1][character]`. The algorithm above explicitly checks this condition and insert the edge between each character from the comparing words to G .

Therefore, the algorithm above correctly computes the relationship between characters of words in ascending order for every L distinct character in the manuscript M .

1.2 Alphabetical Order Validation

Pseudocode

```
ValidAlphaOrder(G):  
    /*  
    1. Goal: check if the given adjacency list of G represents  
    2. I/O:  
        2.1 Input: adjacency list of "alphabetical order"  
        2.2 Output:  
            - If there exists alphabetical order: return true  
            - If there not exists alphabetical order: return false  
    3. How:  
        - application of Zero-indegree Algorithm that finds  
        - in our case, the order is alphabetical. However  
        - If all nodes have zero indegree during the process  
    */  
    // Zero-Indegree algorithm that returns the topological  
    alpha_order <- ZeroIndegree(G) // if $v$ is discovered  
    return alpha_order
```

Runtime Analysis

1. $O(n+m)$: runtime of ZeroIndegree algorithm from the class that iterates through G until either we detect a cycle (or not alphabetical order in this problem) or we traverse through every node in G . Because G has $n=L$ distinct characters where n denote the number of nodes and m the number of alphabetical order candidates from M .

Since the algorithm requires to output only either the alphabetical order if there exists a valid order that does not create a cycle or an exception that says there exists a cycle in the graph G , the algorithm only requires to implement ZeroIndegree algorithm and return its value. Therefore, the final runtime of the algorithm is $O(n+m)$.

Proof of Correctness

lemma1: If G has a topological order, the G is a DAG. Lemma2:

ZeroIndegree(G) outputs either a topological order of G or an exception with a cycle detected. Lemma3: If there is a cycle in graph G , then there exists a node u remaining in G after the execution of ZeroIndegree Algorithm. Claim: If G is in alphabetical order, then G is in topological order.

According to ZeroIndegree algorithm, if it returns the topological order, then

1. it must terminate when it iterated through the entire input graph
2. the input graph G must be empty by the time the algorithm terminates.

First, because alphabetical order does not allow a cycle as it would require a character to precede itself, if something is in alphabetical order, then that is acyclic. If an order is acyclic, then it can be represented as DAG. According to the ZeroIndegree Algorithm, it must remove every node and return a topological order that contains all L distinct characters from G . By the definition of topological order, every edge from one node to another-- $u \rightarrow v$ -- must place u before v . Each edge represents the relationship of the first different character of consecutive word; therefore, every consecutive pair of distinct character is alphabetically ordered. Second, if the algorithm detects a cycle, since there exists no pairs that has a character that precede itself by the algorithm, the algorithm will simply not delete all nodes in G and return "no alphabetical order."

2. Trip planning

Pseudocode

```
TrainPlan(train, s):  
/*  
goal: find the earliest arrival times to each city from s
```

input:

1) train: set of n 4-tuples.

$\text{\$train}(i) = (\text{depcity}(i), \text{arrcity}(i), \text{dep}(i), \text{arr}(i))$

- $\text{\$depctiy}\$$: list of departure cities

- $\text{\$arrcity}\$$: list of arrival cities

- $\text{\$dep}\$$: list of departure time

- $\text{\$arr}\$$: list of arrival time

2) s: starting city

output:

earliestArr: list of earliest arrival time to each ci

* earliest arrival time list includes arrival time to the

How to approach this problem:

- to find the earliest arrival time of each city from

- sort the schedule by departure time (early $\rightarrow$ late)

- to fit the tightest schedule for earliest arrival,

- if the above condition is satisfied, then we just p

*/

// first, sort the $\text{\$train}\$$ list by ascending departur

MergeSortByDep(train) // mergesort algorithm for sort

// now initialize the return table

earliestSchedule <- empty hash table // return value:

// initialize the table with infinity

// $\text{\$train}\$$ is a set of n 4-tuples. so there are n

for $i=0$ in range(n):

 earliestSchedule[train[i][0]] <- ∞

 earliestSchedule[train[i][1]] <- ∞

earliestSchedule[s] <- 0 // first, set starting city

// Greedy algorithm: getting earliest arrival

for (depcity, arrcity, dep, arr) in train:

 if earliestSchedule[depcity] <= dep:

 earliestSchedule[arrcity] <- min(earliestSche

return earliestSchedule

Runtime Analysis

1. $O(n \log n)$: Mergesort is $O(n \log n)$ for all of best, average, and worst case.
2. $O(n)$: initializing earliest schedule hash table with infinity.
3. $O(n)$: greedy algorithm that gets us the earliest arrival time from s to each city.

Therefore, the total runtime of the algorithm is $O(n \log n) + O(n) + O(n) = O(n \log n)$.

Proof of Correctness

For the algorithm to be correct, it must:

1. return an each city's earliest arrival time for all connected cities to the starting city s , and
2. return a list of earliest arrival order that the arrival to a city u must be earlier than or at the same time as the departure from u to another city v .

Proof by Induction

1. Base case: the earliest arrival time to reach to the first input city, s , is 0. Otherwise, ∞ , indicating not reached yet.
2. Inductive Hypothesis: if k train trips are in ascending departure time order, then `earliestSchedule[arrcity]` is the earliest arrival time from s to `arrcity` for k train trips. Considering $k+1$ train trips, if the arrival time to the city u is greater than departure time from the city u , then it is not the earliest arrival. Otherwise, we can catch the connecting trip and the earliest arrival we can get is the $\min(\text{earliestSchedule}[u], \text{arr})$ that the algorithm ensures `earliestSchedule` table to favor the earlier arrival time such that it will minimize the arrival time to any city from the starting city s . Therefore, the algorithm always outputs the correct result